Unit -III: Queue types & operations

Abanggi Kimrang S Momin

21/10/26

Types of Queues

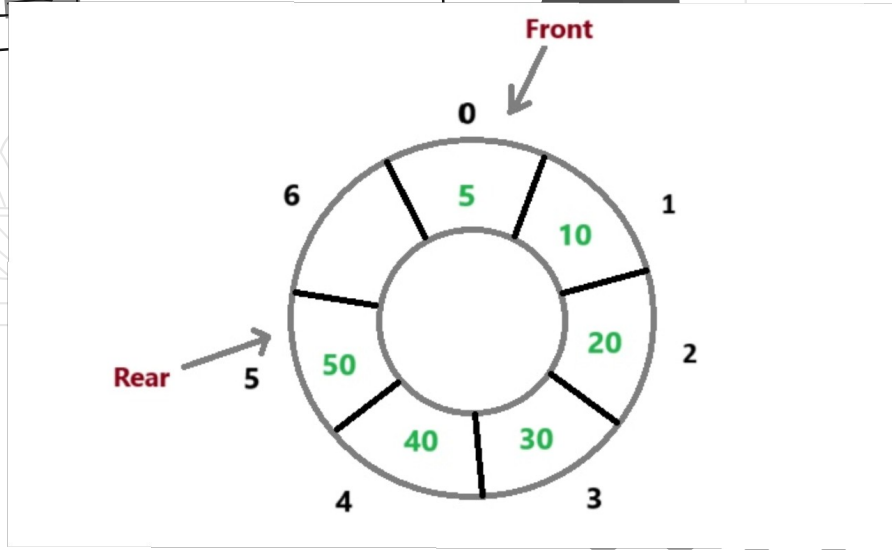
- A queue data structure can be classified into the following types:
  - **Circular queue**
  - **Priority queue**
  - **Deque**



Circular queue

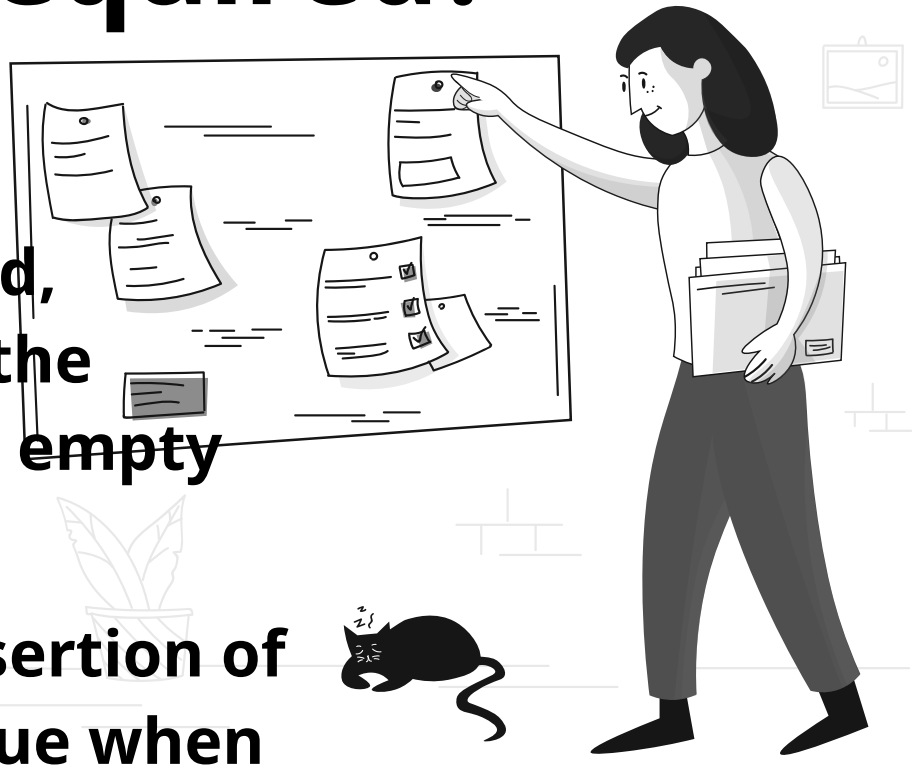
- It is an extended version of a regular queue where the **last element is connected to the first element**. Thus **forming a circle-like structure**.





Why is it required?

- If a **regular queue is full and more items are to be inserted, it cannot be done so even if the front of the queue may have empty spaces**
- **Circular queue allows the insertion of items at the start of the queue when we reach the end of the queue**

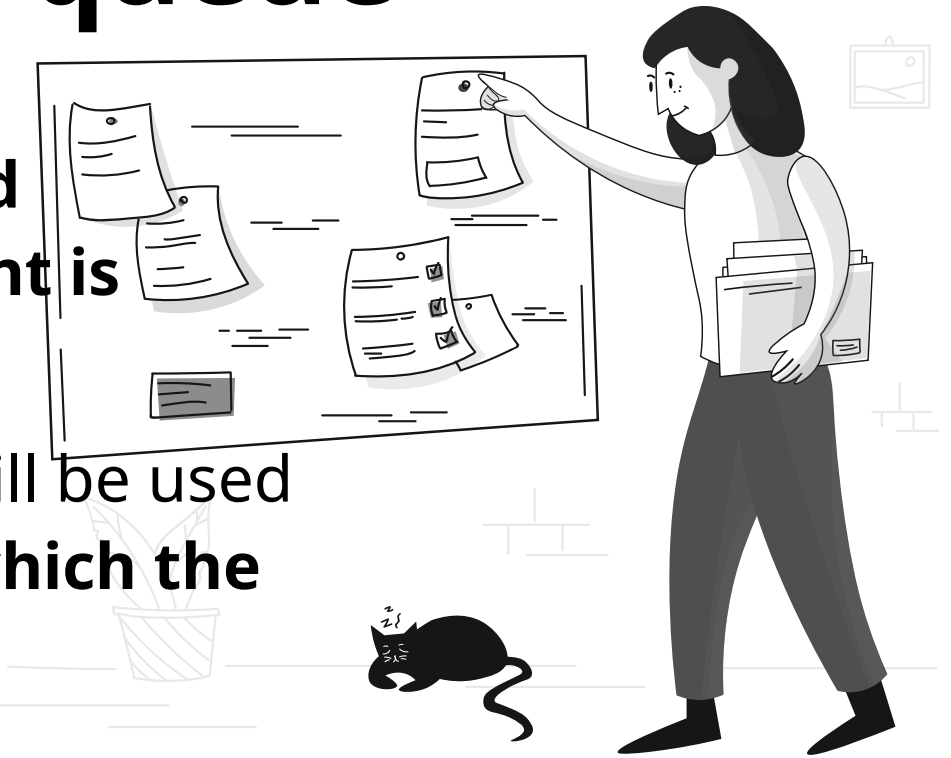


Priority queue in daily life

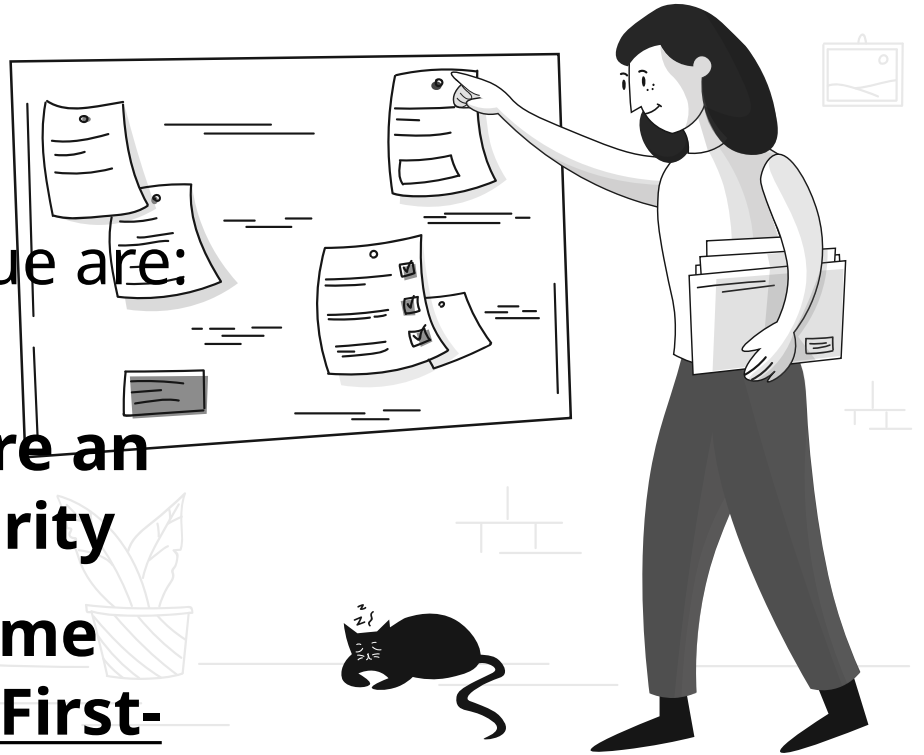


Priority queue

- A priority queue is a **modified queue in which each element is assigned a priority**
- The priority of the element will be used **to determine the order in which the elements will be processed**
- Widely **used in OS for process scheduling**

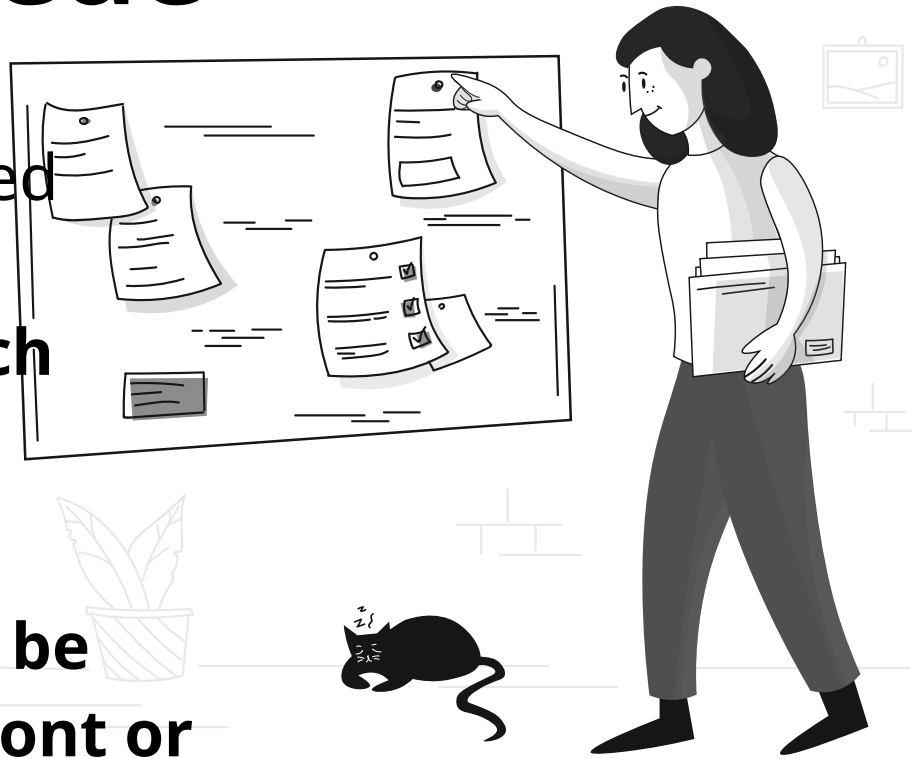


- The general rule of processing the elements of a priority queue are:
  - **An element with higher priority is processed before an element with a lower priority**
  - **Two elements with the same priority is processed on a First-Come-First-Serve (FCFS) basis**



D-Queue

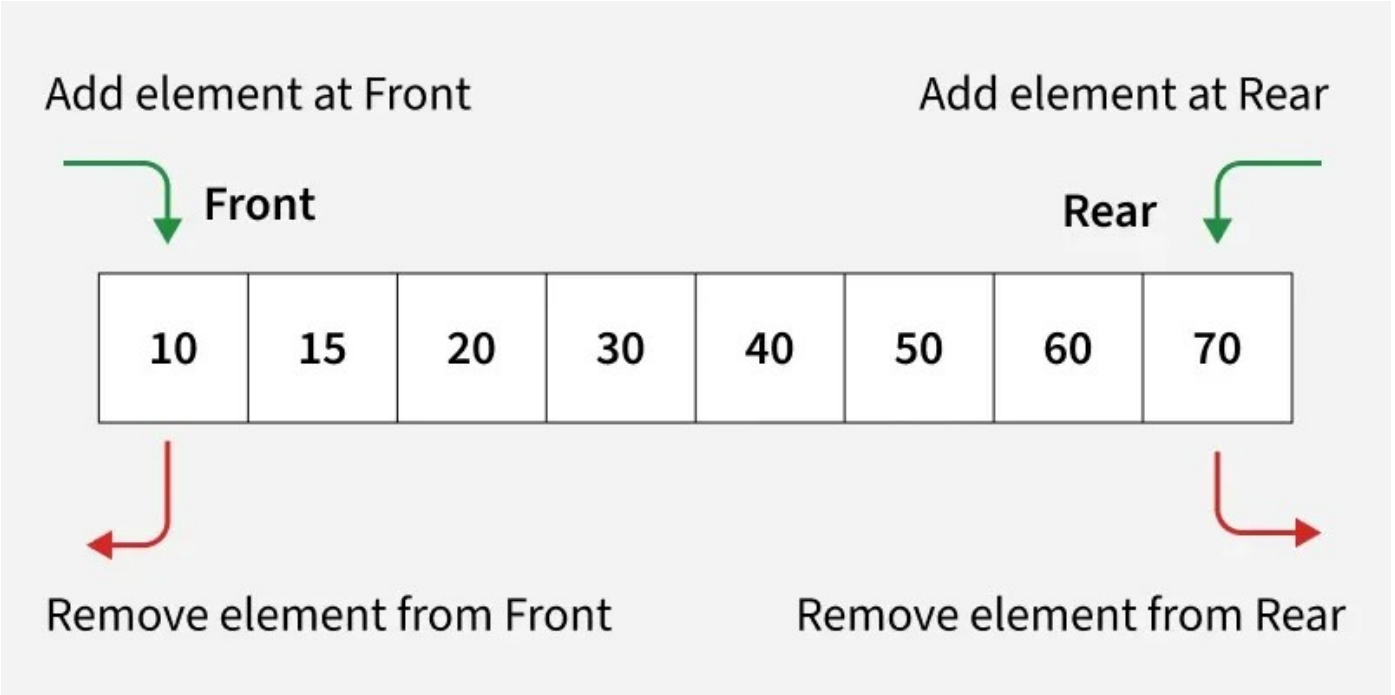
- **D-queue** or **deque** (pronounced as '*deck*' or '*dequeue*') is a **special type of queue in which elements can be inserted or deleted at either end**
- In other words, **elements can be inserted/deleted at either Front or Rear**



- A dequeue **can be implemented using either a circular array or a circular doubly linked list**



Example of Deque



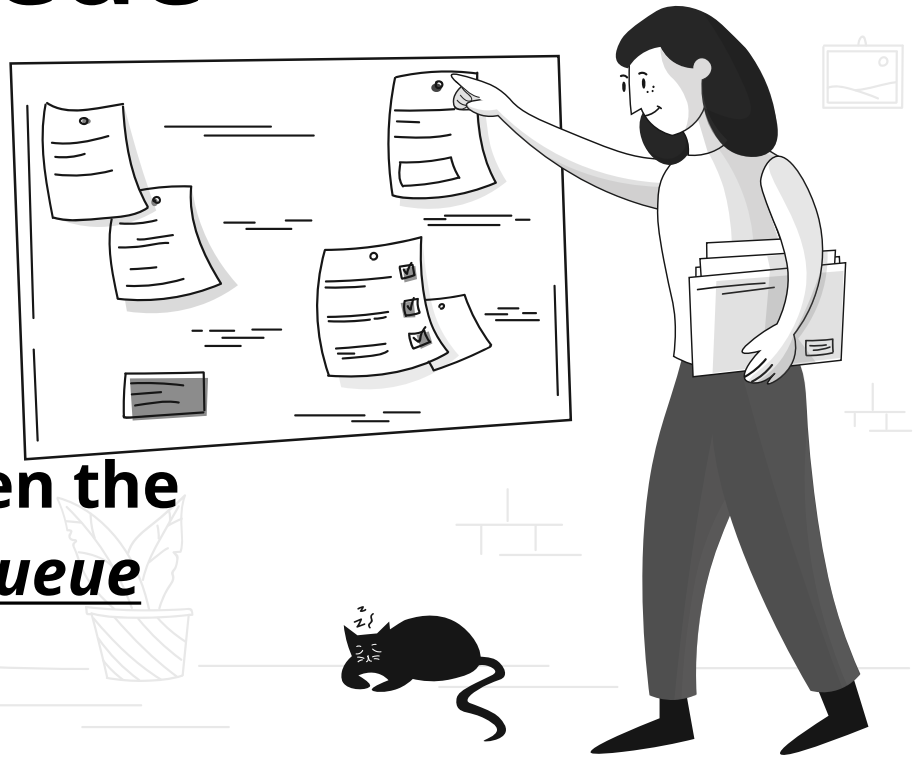
Linear queue operations

- The four main operations on a queue are:
  - **Enqueue**
  - **Dequeue**
  - **Peek**
  - **Display**



Enqueue

- Enqueue operation **inserts an element at the Rear of the queue**
- **If enqueue is attempted when the queue is full, it results in a *Queue Overflow* condition**



Enqueue - *Algorithm*

```
Step 1: IF REAR = MAX - 1
          WRITE OVERFLOW
          GOTO Step 4
        [END OF IF]
Step 2: IF FRONT = -1 AND REAR = -1
          SET FRONT = REAR = 0
        ELSE
          SET REAR = REAR + 1
        [END OF IF]
Step 3: SET QUEUE[REAR] = NUM
Step 4: EXIT
```

Enqueue - *Code*

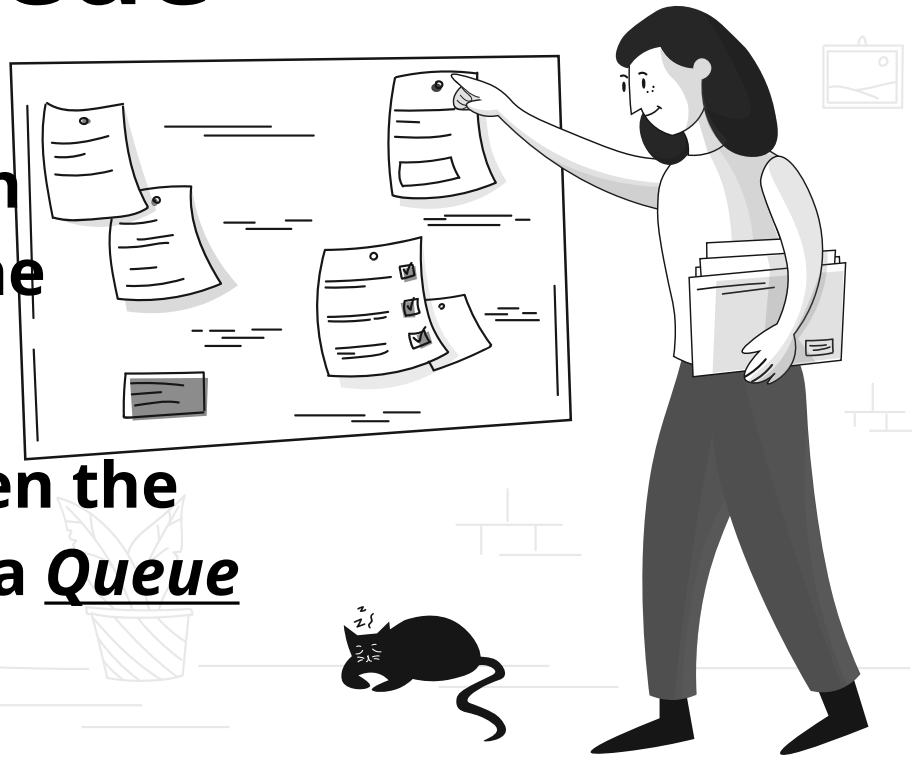
```
void enqueue(int item)
{
  if (rear == MAX-1)
  {
    printf("queue overflow");
    return;
  }

  if (front == -1 && rear == -1)
    front = rear = 0;
  else
    ++rear;

  queue[rear] = item;
}
```

Dequeue

- Dequeue operation **deletes an element from the Front of the queue and returns it**
- **If dequeue is attempted when the queue is empty, it results in a *Queue Underflow* condition**



Dequeue - *Algorithm*

```
Step 1: IF FRONT = -1 OR FRONT > REAR
          WRITE UNDERFLOW
          GOTO Step 2
        [END OF IF]
        SET VAL = QUEUE[FRONT]
        SET FRONT = FRONT + 1
Step 2: EXIT
```

Dequeue - *Code*

```
int dequeue()
{
  int item;

  if (front == -1 || front > rear)
  {
    printf("queue underflow");
    return -1;
  }

  item = queue[front];
  ++front;
  return item;
}
```

Peek

- Peek **returns the first item of the queue or the item at Front**



Peek - *Code*

```c
int peek()
{
  if (front == -1 || front > rear)
  {
    printf("queue empty");
    return -1;
  }

  return queue[front];
}
```

Display

- Display operation simply **traverses the queue and prints every item to the output**



Display - *Code*

```c
void display()
{
  int index;

  if (front == -1 || front > rear)
  {
    printf("queue empty");
    return;
  }

  for (index = front; index <= rear; ++index)
    printf("%d\t", queue[index]);
}
```

That’s it for now

